

COMU

A ground truth engineering assessment of the COMU AI Software Engineering Workspace: architecture, competitive standing against OpenAI Codex and Claude Code, weakness analysis, benchmark framework, and a dependency ordered advancement plan for the next 6 to 18 months.

Repository: D:\COMU · v0.2.4

~20.5k LOC hand written source

22 workspace packages (pnpm)

VS Code extension + local runtime

Contents

Part A • Orientation

- 1. Executive summary
- 2. Headline verdict and scorecard
- 3. Method, scope, and evidence standard

Part B • Architecture assessment

- 4. System architecture and data flow
- 5. Engine and agent loop audit
- 6. Context and reasoning audit
- 7. Tool system audit
- 8. Security, reliability, and permissions
- 9. UI and UX audit

Part C • Comparison and evidence

- 10. Competitive analysis vs Codex and Claude Code
- 11. Claim verification and what is real

Part D • Findings and plan

- 12. Prioritised issue matrix
- 13. Proposed target architecture
- 14. Benchmark and evaluation framework
- 15. Phased advancement roadmap
- 16. Dependency map and parallelisation
- 17. Measurable KPIs
- 18. Differentiation thesis

1. Executive summary

COMU is a genuinely ambitious and, in several places, genuinely well engineered attempt to turn an AI coding agent into a transparent engineering workspace rather than a chat box. The thesis (make visible what the agent is doing, what it found, what changed, what it verified, whether it needs you, and why it finished) is sound and differentiated. The skeleton that expresses that thesis is real: a typed event protocol, a state machine, a plan and verify and repair pipeline, a persistent memory store, a bounded terminal policy, and a seven tab VS Code workspace that most competitors do not attempt.

The problem is that the **intelligence and the wiring have not caught up with the scaffolding**. Under load, the parts that would make COMU actually behave like Codex or Claude Code are either disconnected, stubbed, or template driven. The planner does not plan; it selects one of four fixed templates by keyword. The context engine that gives the product its name is never called on the execution path; the model instead receives an unbounded and growing message history that will overflow the context window on any nontrivial task. The permission contract that promises read only modes is never enforced at runtime. The human in the loop approval infrastructure is fully built and fully unused. The advertised local model runs a different provider with a dummy key. Most seriously, the runtime executes against its own process working directory rather than the user's open workspace, which means the agent very likely operates on the wrong files in a normal install.

The one sentence framing. COMU has built an excellent chassis and dashboard for an autonomous coding agent, but the engine, the fuel line, and the steering column are either placeholder parts or not yet connected; the immediate job is not new features, it is connecting and hardening what already exists so the chassis carries a real engine.

This is not a criticism of ambition; it is a statement of sequencing. The codebase is young (roughly two weeks of commit history at the time of audit) and the quality of the components that *are* finished (the model request manager with retry and backoff, the filesystem safety layer, the terminal command policy, the plan validator, the diagnostics fingerprinting, the memory persistence) is high enough to prove the team can execute. The gap is that a large fraction of the headline capabilities are presentation without a backing implementation. A user who installs v0.2.4 today and points it at a real repository will most likely see an impressive interface narrate a task that either operated on the wrong directory, failed a completion gate it could not satisfy, or silently degraded to a canned response.

What is strong (preserve and build on)

- **Reliability boundary for model calls.** `ModelRequestManager` implements exponential backoff with jitter, a retry budget, a correct non retryable error taxonomy, timeout handling, cancellation propagation, and credential redaction. This is production grade.
- **Filesystem safety.** `resolveAndVerifyPath` uses `path.relative` boundary checks plus realpath based symlink escape prevention, with SHA 256 optimistic concurrency and deterministic unique match edits.
- **Terminal command policy.** Allowlisted executables, destructive and network denial, shell injection character filtering, inline interpreter blocking, environment sanitisation, output redaction, process tree termination, and output truncation.
- **Deterministic scaffolding.** Plan validator with Kahn cycle detection, repair budgeting with duplicate strategy fingerprints, diagnostics fingerprinting, a typed event protocol, and a persistent per workspace memory store with atomic writes.
- **The workspace UI.** A seven tab engineering surface (Activity, Overview, Plan, Changes, Verification, Memory, Workers) with a context drawer and native VS Code diffs. Conceptually ahead of most competitors on transparency.

What is broken or hollow (fix before adding anything)

- **Wrong working directory.** The runtime uses `process.cwd()` as the workspace root and ignores the workspace the extension sends. **CRITICAL**
- **No context management.** Message history grows without bound or compaction; long tasks will fail on context overflow. The named Context Engine is dead code on the execution path. **CRITICAL**
- **Planning is a template selector, not a planner.** Four fixed plans chosen by keyword; the model is never told the plan or the current step. **HIGH**
- **Permission contract not enforced; approvals never raised.** ASK and PLAN modes can still write and execute; the built approval flow is never triggered before mutations. **CRITICAL**
- **Advertised capabilities that are not wired.** Ollama silently runs NVIDIA with a dummy key; the UI mode selector is dropped before it reaches the runtime; CHAT returns a hardcoded greeting with no model call. **HIGH**
- **No model routing or fallback.** The `ModelRouter` interface exists but is unimplemented; retries hit the same provider only. **MEDIUM**
- **Local server without auth.** Express listens on all interfaces with open CORS and no token; anything that can reach the port can drive the agent to read and write files and run commands. **HIGH**

The remainder of this document quantifies each of these, traces the technical root cause, states severity and blast radius, and proposes a concrete, dependency ordered plan. The guiding principle throughout: **COMU does not need a rewrite; it needs its existing parts connected, its stubs replaced with real implementations, and its safety promises made true.**

2. Headline verdict and scorecard

Scores are a considered engineering judgement on a 0 to 10 scale, where 10 is parity with the best current coding agents on that dimension. They weigh both what exists and how reliably it works on a real repository, not what the README claims.

DIMENSION	SCORE	BASIS (OBSERVED UNLESS NOTED)
Repository understanding	 3.0	Recursive JS file search and a tree tool; no semantic index, no symbol graph, no gitignore aware ranking. Model reads files one at a time into unbounded history.
Long context reasoning	 1.5	No compaction, no token budgeting on the wire; the Context Engine is not called. Overflow is a hard failure.
Planning / task decomposition	 2.0	Keyword driven selection of four fixed templates; no model authored plan; plan not fed back to the model.
Autonomous execution loop	 4.5	Real ReAct style loop with a disciplined state machine, cancellation, and limits; but step to model coupling is loose and completion is proxy based.
Tool calling and execution	 6.0	Clean tool abstraction, safe filesystem and terminal tools, OpenAI tool schema mapping and streaming reassembly all work.
Test / verify / repair	 5.0	Multi ecosystem verification and a bounded repair loop exist; diagnosis parsing is JS and TS only; completion gate can misfire.
Git awareness	 4.5	Status, diff, branch, stage, commit, push tools exist; push approval is a self set boolean, not a human gate.
Model routing / fallback	 1.0	Interface declared, never implemented; single provider per task, same provider retries only.
Security / sandboxing	 4.0	Good application level guards; but no OS sandbox, no server auth, permission contract unenforced, approvals unwired.
Reliability of model calls	 8.0	Backoff, jitter, retry budget, error taxonomy, cancellation, redaction. A standout.
Observability	 7.0	Rich typed event stream, SSE replay, and a UI that surfaces state. Best in class ambition; weak persistence.
Resumability / durability	 2.0	In memory task state, cleared 5 minutes after completion; no checkpoint or resume of an in flight task.
UI / UX	 6.5	Ambitious, structured, transparent; hurt by a shipped debug bar, tab overload in a narrow rail, decorative controls, and no pre write approval.
Correctness on a real repo today	 2.0	The working directory defect alone puts baseline correctness at risk in a normal install.

7

CRITICAL BLOCKERS

9

HIGH PRIORITY ISSUES

~20.5k

LOC REAL SOURCE

36

TEST FILES / ~325 CASES

4

PROVIDERS LISTED / 2 REAL

0

MODEL AUTHORED PLANS

Reading guide for the team. Do not start Phase 2 features (semantic index, agent teams, CI integration) until the Phase 0 correctness and safety defects in section 12 are closed. Building capability on top of the working directory defect, the unbounded context, and the unenforced permission contract will multiply technical debt and produce demos that do not survive contact with a real repository.

3. Method, scope, and evidence standard

The assessment was performed by direct static inspection of the repository at `D:\COMU` (version 0.2.4). Every core module was read in full: the orchestrator and kernel, the intent router and task contract, the planner and plan state manager and validator, the model request manager and OpenAI compatible provider and capability profiles, the context engine and working set, the verification, repair, and diagnostics engines, the diff engine surface, the memory storage layer, the tool core executor and parser and registry, the filesystem, terminal, git, search, validation, and web docs tools, the Express runtime server, the extension host (activation, runtime client, SSE client, health monitor, server process manager, chat provider, diff viewer), and the webview (HTML shell, state rendering, event normalisation).

Evidence tags used throughout

OBSERVED read directly in the source; behaviour follows from the code as written. **INFERENCE** an engineering deduction from the code that a live run would confirm. **UNVERIFIED** cannot be settled without a live run or external confirmation; the evidence needed is stated.

Severity scale

LABEL	MEANING	ACTION WINDOW
CRITICAL	Breaks core correctness or safety; the product misbehaves or is unsafe in a normal install.	Before any new feature work.
HIGH	A headline capability is hollow, or a real security or reliability weakness exists.	Immediate next cycle.
MEDIUM	A meaningful limitation that constrains real use or scale.	Stabilisation to modernisation window.
LOW	Polish, consistency, or hygiene.	Opportunistic.
EXPERIMENTAL	A differentiation opportunity, not a defect.	After the foundation is sound.

Scope limits

Findings are static and architectural. The agent was not executed against a live model or a real task, so runtime behaviours (the actual context overflow point, the actual working directory VS Code assigns to the extension host, real end to end latency and token cost) are marked **INFERENCE** or **UNVERIFIED** with the confirming experiment named. Third party comparisons rely on publicly documented behaviour of Codex and Claude Code as of September 2026 and are labelled accordingly.

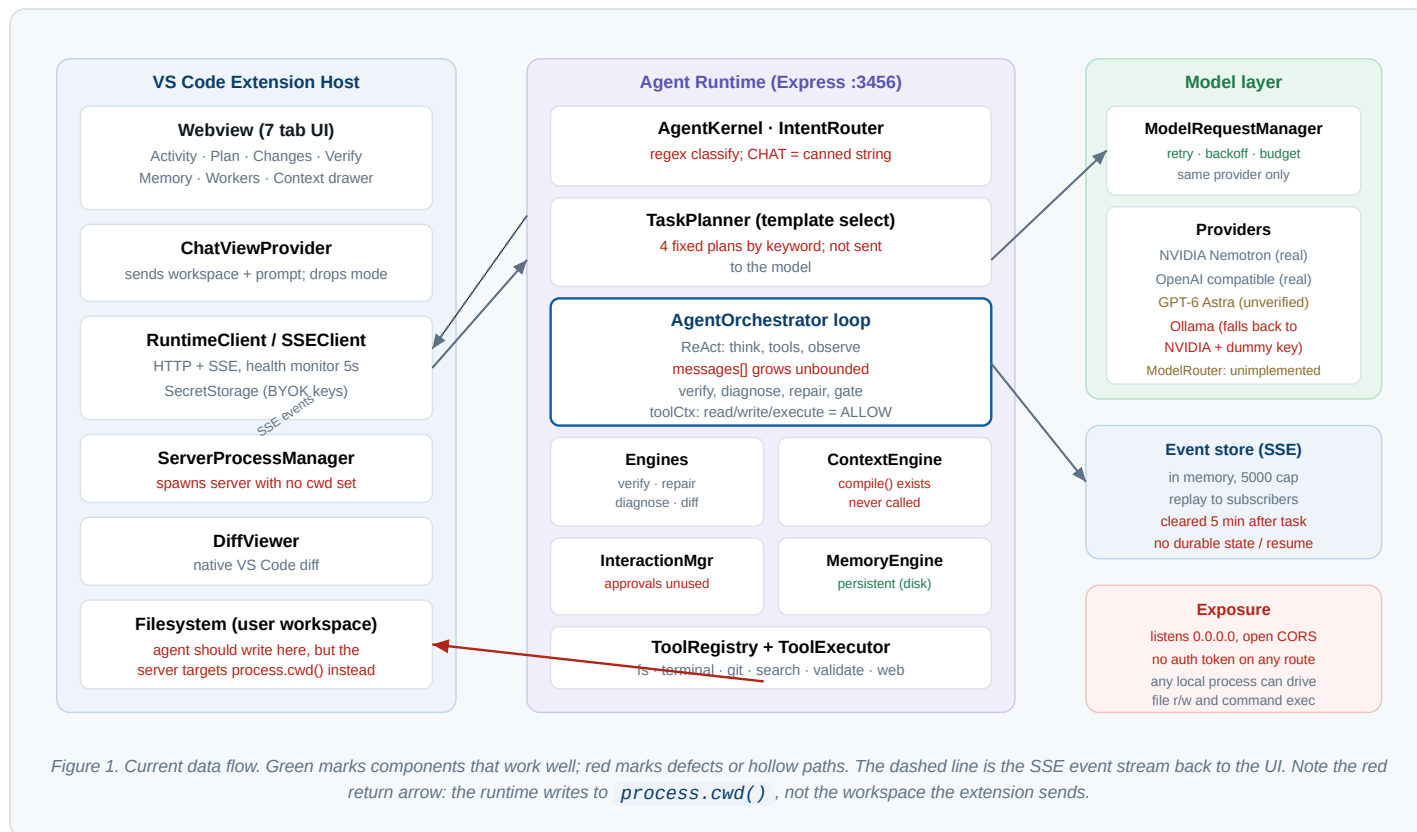
Real source inventory (excluding bundled test fixtures)

The repository appears to hold about 77k lines under the extension package, but roughly 68k of that is bundled VS Code and GitHub Copilot SDK type stubs under `.vscode-test`. The true hand written surface is about 20.5k lines: approximately 14.7k of TypeScript across packages, apps, providers, and tools, plus about 5.7k in the webview assets (`main.js` 2.2k, `style.css` 3.1k, `index.html` 0.4k).

LAYER	PACKAGES	APPROX LOC	ROLE
Agent brain	<code>agent-core</code> , <code>planning-engine</code> , <code>context-engine</code>	~3.3k	Orchestrator, kernel, intent router, planner, working set.
Quality engines	<code>verification-engine</code> , <code>repair-engine</code> , <code>diagnostics-engine</code> , <code>diff-engine</code>	~1.1k	Verify, diagnose, repair, changeset.
Model layer	<code>model-core</code> , <code>providers/nvidia</code>	~1.8k	Request manager, OpenAI compatible provider, profiles, NVIDIA.
Memory + protocol	<code>memory-engine</code> , <code>protocol</code> , <code>shared</code>	~1.7k	Persistent memory, typed events, errors.
Tools	<code>tool-core</code> , <code>filesystem</code> , <code>git</code> , <code>terminal</code> , <code>search</code> , <code>validation</code> , <code>web-docs</code>	~3.0k	Tool contract and the tool suite.
Runtime + extension	<code>agent-runtime</code> , <code>vscode-extension</code>	~3.4k + webview	Express server, SSE, extension host, seven tab UI.

4. System architecture and data flow

COMU is a pnpm monorepo that ships as a VS Code extension plus a local agent runtime. The extension host renders a webview and spawns a Node Express server (port 3456) that owns the orchestrator, the tool registry, and the engines. Communication is HTTP for commands and Server Sent Events for the live event stream, with an in memory event store that replays history to a late subscriber.



What the architecture gets right structurally

- **Process separation.** Putting the agent in a separate Node process rather than inside the extension host is the correct choice; it isolates crashes, avoids blocking the UI thread, and would allow a remote runtime later.
- **Event sourced UI.** A typed event stream with replay is the right substrate for the transparency thesis and for future observability and audit.
- **Clean package boundaries.** Engines are separated behind interfaces (planner, verification, repair, memory, tools), which makes eventual replacement of stubbed internals possible without touching the orchestrator surface.
- **Per task orchestrator instance.** A fresh orchestrator per task avoids cross task state leakage in the request manager binding, a subtle correctness win.

Where the architecture is fragile

- **The workspace root is never plumbed through.** The extension knows the workspace and sends it; the server discards it. This single missing line of coupling undermines everything downstream.
- **The Context Engine sits beside the loop, not inside it.** The orchestrator talks to the model directly with a raw message array; the compiled context and working set are UI decorations rather than prompt inputs.
- **State lives only in memory.** There is no durable task journal, so a runtime restart loses everything and there is no resume.
- **One provider per task, chosen by string matching on a model id.** There is no capability aware router and no fallback across providers.

5. Engine and agent loop audit

The heart of COMU is `AgentOrchestrator` (about 1.2k lines) driven by `AgentKernel`. The design is a plan gated ReAct loop with a formal state machine, a verification and repair sub cycle, and a completion gate. The bones are good. The problem is that several of the load bearing joints are either loose or ornamental.

5.1 The state machine (a genuine strength)

OBSERVED The orchestrator enforces an explicit transition table across fourteen states (IDLE, STARTING, CLASSIFYING, ANALYZING, PLANNING, THINKING, TOOL_CALLING, OBSERVING, VERIFYING, DIAGNOSING, REPAIRING, WAITING_FOR_USER, and the terminals COMPLETED, FAILED, CANCELLED, LIMIT_REACHED). Illegal transitions throw, and mode aware guards block TOOL_CALLING in CHAT and PLAN and block REPAIRING outside AGENT. This is more disciplined than most agents, which run an untyped while loop. Preserve it.

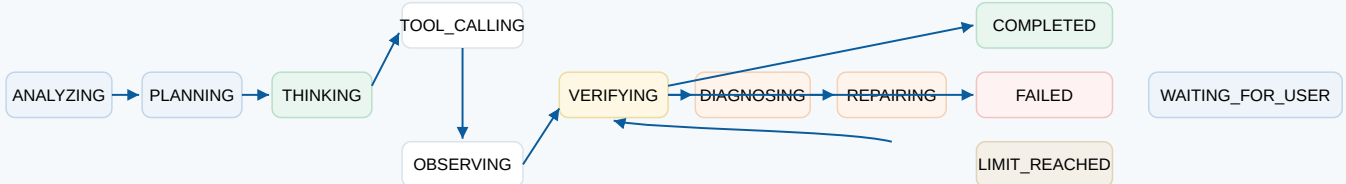


Figure 2. The core execution states. The verify, diagnose, repair triangle is the automated remediation sub cycle. WAITING_FOR_USER exists but is only reached from the AMBIGUOUS classification path, not from a mid task approval request.

5.2 The plan is decorative **HIGH**

OBSERVED `TaskPlanner.createPlan` calls `analyzeTask`, which is pure keyword matching on the prompt, and returns one of four hardcoded step templates: a single INVESTIGATE step for questions, a three step INVESTIGATE, IMPLEMENT, VALIDATE for simple and feature tasks, and a five step reproduce, diagnose, repair, revalidate template for anything that mentions test, fail, fix, or bug. No model call participates in planning. Every feature request, regardless of content, produces the identical three step plan with identical titles like "Implement requested engineering changes."

OBSERVED Worse, the plan is never shown to the model. The orchestrator loops, and on each iteration it picks the current plan step for its own bookkeeping, but the message it sends to the model is always the original user prompt plus the growing history. A plan step is marked COMPLETED whenever the model returns a turn with no tool calls. So the plan is a UI artifact: it drives status badges, not the model's behaviour, and completion is inferred from the proxy signal "the model stopped calling tools" rather than from any real check that the step's objective was met.

Consequence. Task decomposition, the single most important lever for multi file and long horizon work, does not exist. The agent runs a generic ReAct loop and hopes the model self organises. On a task that genuinely needs staged decomposition (touch five files, keep an interface consistent across them), there is no structure guiding the model and no per step verification of intent, only a global gate at the end.

Root cause. Planning was implemented as deterministic scaffolding to satisfy the plan UI, not as a reasoning step. **Fix direction.** Replace `analyzeTask` with a model authored plan (a dedicated planning call that emits structured steps validated by the existing, and genuinely good, `PlanValidator`), then inject the current step and its acceptance criteria into the system prompt for each execution turn so the plan actually steers the model. The validator, state manager, and repair mutation logic can be kept as is; only the plan source and the plan to prompt binding need to change.

5.3 Completion is proxy based, and the gate can misfire **MEDIUM**

OBSERVED The completion gate requires that required verification passed, no critical checks failed or were unavailable, workspace integrity is verified, no interaction is pending, limits are respected, and the changeset is valid. This is a thoughtful invariant set. But two problems follow. First, an AGENT task with real changes must pass verification to complete, and verification requiredness is decided by heuristics (see 6). If the project has no runnable tests, or the validator command is not on the allowlist (see 7.4), a required check becomes UNAVAILABLE and the task is reported FAILED even though the work may be correct. Second, for a task with no file changes the gate trivially passes with all optional checks, so a task that should have changed files but did not is reported as success.

5.4 No context budget in the loop **CRITICAL** (detailed in section 6)

OBSERVED Each iteration appends the assistant turn and every tool result to a `messages` array that is sent in full on the next call. Tool results are JSON stringified with no history level truncation, and the filesystem read limit is 1 MB per file. There is no tokeniser, no compaction, and no summarisation. On a task of even moderate length the request will exceed the model's context window and the provider will return a 400, which the request manager correctly classifies as non retryable, so the task fails hard. This is the ceiling on task length today.

5.5 Tool execution is sequential and the contract is not enforced **HIGH**

OBSERVED When the model returns several tool calls, the orchestrator executes them one at a time in a `for` loop with `await`; there is no parallelism even for independent read only calls. More importantly, the loop calls `executor.execute()` directly, which checks only the hardcoded `toolCtx.permissions` (read, write, execute all ALLOW; network DENY). The `processModelToolCall` path that would enforce the `TaskContract` capabilities is never used. So the read only promise of ASK and PLAN modes is not enforced at the tool layer; only the state machine's `TOOL_CALLING` guard blocks PLAN, while ASK can still reach tools with write and execute allowed.

5.6 Mutation integrity handling is careful, sometimes too careful **MEDIUM**

OBSERVED Around each mutating tool call the orchestrator reads the target file before and after to compute a baseline and final hash, and records the change in the changeset. This is a genuinely good integrity mechanism. However, when a mutating tool throws but the file state changed, the orchestrator fails the entire task with `WORKSPACE_STATE_CHANGED_AFTER_TOOL_FAILURE` rather than surfacing the error to the model for recovery. A more forgiving policy would let the model see the error and decide, reserving a hard fail for genuinely unverifiable states. Also, the double read per mutation doubles filesystem tool traffic and inflates the event stream.

5.7 Subagents: bounded but shallow **MEDIUM**

OBSERVED `delegate_subtask` supports RESEARCH and VERIFICATION workers only, at depth one (no recursion), max two per task, with tiny budgets (five steps, five tool calls, thirty seconds). Workers are read biased. This is safe and prevents runaway fan out, but it is not the sub agent orchestration the README's "Workers" tab implies: there is no parallel implementation worker, no planner and executor split, no aggregation of independent findings, and delegation blocks the parent (the parent awaits the worker synchronously), so there is no real concurrency.

5.8 Repair loop: the best engineered engine

OBSERVED The repair engine is the strongest of the automated engines. It fingerprints failures and repair strategies, refuses a repair strategy identical to one that already failed for the same failure signature (a real defence against thrashing), and enforces budgets on attempts, validation runs, target files, and time. The plan mutation that injects diagnose, repair, revalidate steps is validated before it is applied. This is the pattern the rest of the system should aspire to. Its main limitation is upstream: the diagnosis it consumes is only as good as the evidence parser, which is JS and TS only (see 6.3).

6. Context and reasoning audit

6.1 The Context Engine is not on the execution path CRITICAL

OBSERVED `packages/context-engine` contains a real `ContextEngine.compile()` that assembles active file, recently inspected files, open files, diagnostics, search results, and a repository map under a character budget. It is well structured. It is also never called by the orchestrator or the runtime. The only context artifact the orchestrator uses is `WorkingSetManager`, and it uses it purely to emit events for the context drawer in the UI. The model itself receives none of it; it gets the raw growing message array. So the product's namesake capability, context engineering, does not touch the model.

Blast radius. Two headline claims collapse at once: the "Context Engine and WorkingSet" capability and the "up to 1,050,000 tokens" support. The context window figure is a provider profile constant, not a managed budget; nothing in COMU curates, ranks, compresses, or bounds what is sent, so a large window would simply be filled with unpruned history until it overflows.

Fix direction. Make `compile()` the single source of the prompt. Introduce a real token budget (a tokeniser or a calibrated heuristic per provider), rank context by relevance and recency, summarise or drop the oldest tool results, and keep a compact rolling task state (goal, plan, current step, key findings, open questions) that survives compaction. This is the highest leverage single change for capability after the working directory fix.

6.2 No repository intelligence MEDIUM

OBSERVED Repository understanding is a recursive JS directory walk (`get_workspace_tree` , ignoring a fixed set like `node_modules`) plus a recursive text search backend (no ripgrep, no gitignore awareness beyond the fixed ignore list). There is no symbol index, no import graph, no embeddings, no definition or reference lookup. The model must discover structure by reading files, which is both slow and context hungry, and it scales poorly on large repositories.

6.3 Diagnosis is JS and TS only MEDIUM

OBSERVED The evidence extractor's regexes match TypeScript compiler errors, Jest, Vitest, and Mocha test output, and JS style stack frames. For Python, Go, or Rust failures, the extractor finds no affected files and the classifier falls back to generic categories. Since repair scope validation depends on affected files, the diagnose to repair loop is effectively a JS and TS feature even though validation command resolution supports several ecosystems (see 7.4). This is a coherence gap between the verify layer (multi language) and the diagnose layer (single language family).

6.4 Memory is real, and disconnected from context assembly

OBSERVED The memory engine persists per workspace JSON under the OS app data directory with atomic writes, corruption backup, trust levels, and episode logging. The orchestrator queries it at task start and injects the top matches as supplementary text, and records lessons and episodes on completion. This is a legitimately useful subsystem. Its limits: retrieval is lexical rather than semantic, and memory is a one shot prefix at task start rather than a live participant in per turn context assembly, so a lesson that becomes relevant mid task is not resurfaced.

REASONING CAPABILITY	STATE	EVIDENCE
Token budgeting on the wire	Absent	No tokeniser; messages sent in full.
Context compaction / summarisation	Absent	No history pruning anywhere in the loop.
Relevance ranked file selection	Built, unused	<code>compile()</code> ranks, but is not called.
Repository map	Built, unused	Tree tool exists; produced in <code>compile()</code> only.
Semantic code search / index	Absent	Recursive text scan only.
Persistent project memory	Present	Disk backed, trust scored, episodic.
Rolling task state across compaction	Absent	No compact state object; relies on full history.

7. Tool system audit

The tool layer is, alongside the model request manager, the most mature part of COMU. The abstraction is clean (a registry, an executor with timeout and permission checks, a canonical parser that distinguishes text from malformed and valid tool calls), and the individual tools are careful. The weaknesses are at the edges: an allowlist that is too narrow for real projects, two divergent command execution paths, and JS centric assumptions.

7.1 Filesystem tools (strong)

OBSERVED `resolveAndVerifyPath` resolves against the workspace root, rejects paths that escape via `path.relative` (not the naive prefix test), and additionally resolves symlinks with `realpath` to block symlink escapes. `read_file` returns content, a SHA 256 hash, size, and line count with optional line ranges and a size cap. `edit_file` requires the search text to match exactly once (fails on zero or multiple matches) and supports an expected hash for optimistic concurrency. `create_file` refuses to overwrite; `write_file` supports expected hash. This is correct, defensive, and deterministic. Two gaps: everything is UTF 8 (a binary or image file would be mangled), and there is no delete or move or rename tool, so certain refactors are awkward.

7.2 Terminal tool (strong, with two sharp edges)

OBSERVED Commands go through a policy: an allowlist of development executables (node, npm, pnpm, yarn, tsc, eslint, prettier, jest, mocha, vitest, python, pytest, go, cargo, rustc, java, javac, mvn, gradle), denial of destructive (rm, dd, format, shutdown) and network (curl, wget, ssh) executables, shell injection character filtering, inline interpreter blocking (node -e, python -c), a workspace boundary check on the working directory, environment variable sanitisation for secrets, output redaction, output truncation, timeouts, and process tree termination. This is a serious, well thought out sandbox at the application level.

Sharp edge 1: shell true on Windows. **OBSERVED** The process manager spawns with `shell: process.platform === 'win32'`. On Windows the command is handed to cmd.exe. The injection filter blocks `; & | > < $ ` $(` but not the Windows specific `%VAR%` expansion or the caret escape. The allowlist limits the blast radius, but running through a shell weakens the otherwise strong no shell posture.

Sharp edge 2: boundary prefix test. **OBSERVED** The terminal working directory check uses `normalize(cwd).startsWith(normalize(root))`, which admits a sibling directory that shares a name prefix (for example a root of `proj` and a cwd of `proj-evil`). The filesystem tools use the correct `path.relative` test; the terminal tool should adopt the same helper.

OBSERVED A deeper product limitation: the allowlist is too restrictive for real repositories. There is no `make`, no `bash` or `sh`, no `docker`, no arbitrary project scripts, and no shell operators (so `cd x && npm test` is impossible; the agent must use the `cwd` argument). Many real projects cannot be built or tested within this allowlist. A capability that many tasks need, running the project's own scripts, is unavailable, which will push users toward manual steps.

7.3 Git tools (functional; the approval is theatre) **HIGH**

OBSERVED Status, diff, create branch, stage, commit, and push tools exist and validate their inputs (for example the push tool rejects a malformed remote name and refuses to push from detached HEAD). But `git_push` claims to "strictly require explicit human approval" while the approval is an `approved: boolean` field in the tool arguments, which the model supplies. The model can set it to true itself; there is no human in the path. This is self asserted approval, not a gate. Separately, the git tools build command plans and call the process manager directly, bypassing `CommandPolicy` (git is not on the allowlist), so there are two command execution paths with different safety properties. The git path is lower risk because its argument arrays are constructed, not model authored, but the inconsistency should be resolved.

7.4 Validation tools (multi language resolver, single language policy) **MEDIUM**

OBSERVED The validators (`run_tests`, `run_build`, `run_linter`, `run_typecheck`) detect the project type (Node, TypeScript, Python, Go, Rust, Java) and resolve appropriate commands: npm and pnpm and yarn scripts or tsc for Node and TS, pytest and flake8 and mypy for Python, go test and build and golangci-lint for Go, cargo for Rust. Good intent. But two coherence bugs: Java is detected yet has no command mapping (always UNAVAILABLE), and the Python and Go linters and type checkers the resolver emits (flake8, mypy, golangci-lint) are not on the terminal `CommandPolicy` allowlist, so they are denied at the policy layer. The result is that non JS verification silently degrades: either UNAVAILABLE (which can fail a required gate) or DENIED.

7.5 Search and web docs

OBSERVED Search is a hand rolled recursive Node scan (a code comment notes ripgrep could be added), with read capability. It will be slow on large repositories and lacks gitignore awareness. Web docs fetches over HTTPS with a domain policy (HTTPS only, allowlisted domains) but is registered with the `execute` capability rather than `network`, which is why it works despite the orchestrator hardcoding network to DENY; a small miscategorisation that happens to be load bearing.

7.6 Tool system summary

TOOL	CAPABILITY	ASSESSMENT	PRIORITY GAP
read_file / write_file / edit_file / create_file	read / write	Strong, safe, deterministic	No binary handling; no delete or move.
list_directory / get_workspace_tree	read	Fine	Fixed ignore list, not gitignore.
search_text	read	Works, slow	No ripgrep; no semantic search.
execute_command (terminal)	execute	Strong policy, too narrow	shell:true on Win; prefix boundary; no project scripts.
git_* (six tools)	execute	Functional	Self asserted push approval; bypasses policy.
run_tests / build / lint / typecheck	execute	Multi language resolve	Policy denies py/go linters; Java unmapped.
web_docs	execute (should be network)	Works	Capability miscategorised.

8. Security, reliability, and permissions

COMU shows strong instincts at the tool level (allowlists, path safety, secret sanitisation) but the perimeter and the authorisation model have real gaps. The pattern is consistent: the low level guards are good, the system level controls are missing.

8.1 Perimeter: an unauthenticated local server HIGH

OBSERVED The runtime is an Express app started with `app.listen(port)` (default bind is all interfaces) with `cors()` wide open and no authentication on any route. The routes create tasks, cancel them, stream events, fetch diffs, and read and write workspace memory. A task, once created, drives file reads and writes and command execution in the workspace. Therefore any local process, and on a permissive network any host that can reach port 3456, can instruct COMU to read files, modify files, and run allowlisted commands. The extension sets an `X-NVIDIA-API-KEY` header, but the server never checks it, so it is not auth.

Fix direction. Bind to `127.0.0.1` only; generate a per session bearer token in the extension, pass it to the spawned server via environment, and require it on every route; scope CORS to the webview origin. This is a small, contained change with large risk reduction.

8.2 Authorisation: the contract and the approvals are both inert CRITICAL

OBSERVED Two authorisation mechanisms are designed but not enforced. First, the `TaskContract` assigns read only capabilities to ASK and PLAN modes, but the orchestrator never validates tool calls against the contract (it bypasses the enforcing path), and it hands every tool a context with write and execute set to ALLOW. Second, `InteractionManager.requestApproval` is fully implemented and wired end to end (runtime endpoint, extension relay, webview response handler), yet the orchestrator never calls it. So there is no approval before a file write or a command run; the agent auto executes everything and the user reviews after the fact, if at all.

Why this matters. The README's transparency thesis explicitly promises "Interactive Human in the Loop Cards" and the ability to "approve actions." The infrastructure to deliver that promise exists and is good; only the trigger is missing. Until the orchestrator raises an approval interaction before mutating or executing (at least in an "ask" autonomy level), COMU is a fully autonomous writer wearing the interface of a supervised one. This is both a safety gap and a trust gap.

8.3 No OS level sandbox MEDIUM

INFERENCE All isolation is in application code. There is no OS sandbox (no seccomp or Landlock on Linux, no Seatbelt on macOS, no job object on Windows), unlike Codex, which sandboxes at the OS level. The allowlist and boundary checks are good defence in depth, but a bug in any tool, or the intentionally permitted `shell: true` on Windows, executes with the full privileges of the user running VS Code. For an autonomous agent that runs commands, OS level confinement should be the eventual target.

8.4 Reliability posture

CONCERN	STATE	DETAIL
Model call resilience	Strong	Backoff, jitter, retry budget, timeout, cancellation, error taxonomy.
Cross provider fallback	Absent	Router unimplemented; only same provider retries.
Task durability / resume	Absent	In memory event store, cleared 5 min post task; restart loses state.
Context overflow handling	Absent	Overflow becomes a non retryable 400; hard fail.
Long running tasks	Capped	5 minute execution cap, 30 steps, 100 tool calls per task.
Cancellation	Strong	Abort checks throughout the loop and in tools; process tree kill.
Secret handling	Good	SecretStorage in extension; env and output redaction; keys never in webview.
Workspace integrity	Good	Hash based change verification; optimistic concurrency on writes.

OBSERVED The 5 minute cap directly contradicts the "long running autonomous development" ambition; realistic multi file tasks routinely exceed five minutes once tests run. Durability and resume are the prerequisites for genuinely long tasks, and both are absent.

9. UI and UX audit

The webview is the most complete expression of COMU's vision and its strongest differentiator. It is a genuine engineering workspace: seven tabs (Activity, Overview, Plan, Changes, Verification, Memory, Workers), a context drawer with the working set, a mode and model selector, native VS Code diffs, and a clean BYOK settings surface with a security explanation. Compared with a plain chat panel, this is ambitious and, in places, delightful. The problems are of two kinds: controls that look functional but are not, and information architecture that does not fit the medium.

9.1 The end to end experience, step by step

STAGE	WHAT HAPPENS TODAY	ISSUE
Launch	Sidebar opens; a "COMU Boot" diagnostic bar shows HTML, JS, DOM, Render, Host, Runtime probes.	LOW A developer debug artifact is shipped in the production UI, permanently occupying the top strip.
Onboard	BYOK card offers NVIDIA, other provider, or local Ollama; suggested workflow chips.	Good. But the Ollama path leads to a provider that does not actually run locally (see section 11).
Start a task	Composer with a Mode selector (Auto, Agent, Plan, Ask, Chat) and a Model selector.	HIGH The Mode selection is never sent to the runtime; the backend reclassifies by regex, so the control is decorative.
See the plan	Plan tab shows step badges (pending, active, completed).	The plan is a fixed template (section 5.2); it does not reflect a real decomposition of the task.
Watch execution	Activity timeline streams normalised events; status pill; live badges.	Strong. This is the best part of the product.
Approve actions	No approval prompt appears; files are written and commands run autonomously.	CRITICAL The promised human in the loop cards never appear; review is post hoc.
Review diffs	Changes tab lists modified files; clicking opens a native VS Code diff per file.	MEDIUM Per file only; no aggregate review, no per hunk accept or reject, and changes are already on disk.
Handle errors	Failure banner with a reason; verification tab shows checks.	Decent. But a spurious gate failure (section 5.3) reads as a real failure to the user.
Interrupt	Stop button cancels; cancellation is handled well throughout.	Good.
Chat	Chat mode returns a fixed greeting, no model call.	HIGH The conversational affordance does nothing.
Resume	No resume; task state is discarded after completion or restart.	MEDIUM No history, no re open, no continue.

9.2 Information architecture: seven tabs in a narrow rail MEDIUM

INFERENCE Seven horizontal tabs with icon, label, and badge will crowd or wrap in a typical VS Code sidebar width (roughly 300 to 400 px). The signature surface (Activity) competes for the same top strip as six siblings, several of which are empty most of the time (Workers is empty unless a subagent runs; Memory is often empty). The result is navigational overhead to reach the two tabs that matter during a task (Activity and Changes).

9.3 Concrete UI proposals

Collapse to a primary stream plus a rail

Make Activity the persistent primary surface with the plan rendered inline as a collapsible header (plan steps as a progress ribbon above the stream). Move Overview, Verification, Memory, and Workers into an on demand "details" drawer or a secondary row that appears only when populated. Keep Changes as a first class tab because review is frequent.

Aggregate diff review with staging

Add a "review all" view that lists every changed file with additions and deletions counts and a combined diff, plus per file and ideally per hunk accept or reject, and a "keep working directory clean until approved" option so review can precede write for users who want it.

Progress and cost transparency

Inline approval cards

When autonomy is set to "ask," render an approval card inline in the Activity stream before a write or a command: show the diff preview or the exact command, with Approve, Approve for session, Edit, and Deny. This uses the existing InteractionManager path; it needs the orchestrator trigger and a card component.

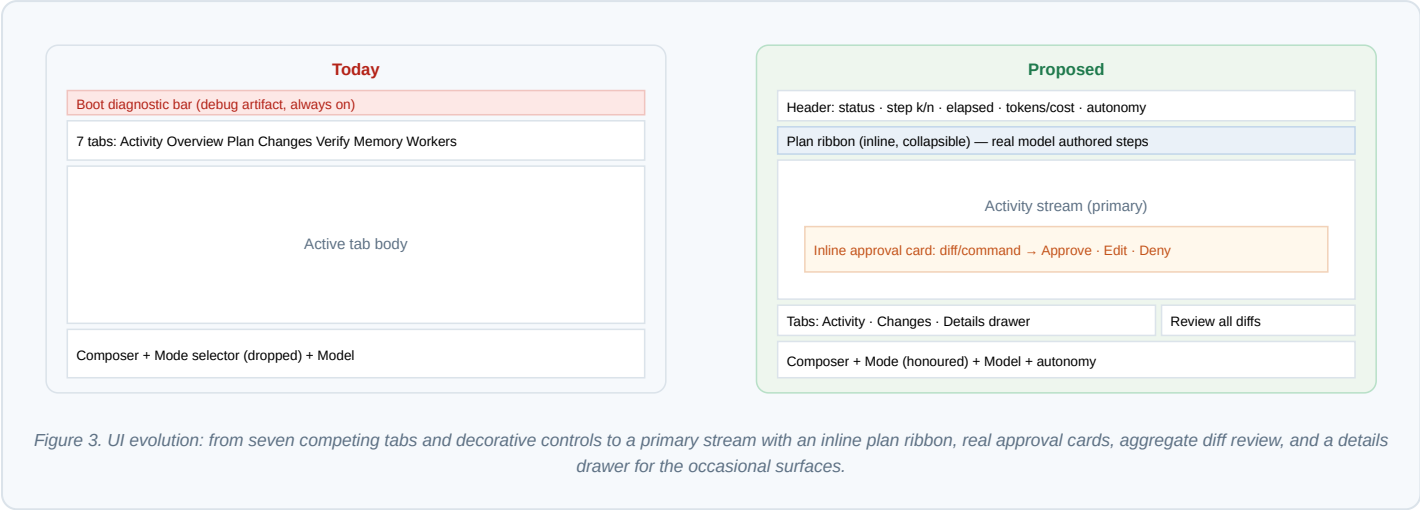
Make the controls real

Send the selected Mode to the runtime and honour it (skip reclassification when the user chose explicitly). Remove or dev gate the Boot diagnostic bar. Add an autonomy selector (read only, ask before write, auto) that maps to the permission contract.

Keyboard and accessibility

Surface step count, elapsed time, and cumulative token and cost estimates (the provider already returns usage) in the header, so the user can see budget burn on long tasks. Add a per task activity log export.

Beyond Enter, Shift+Enter, and Esc, add tab switching shortcuts, an approve or deny shortcut for pending cards, and focus management for the stream. Replace emoji only icons with labelled controls for screen readers; the ARIA tablist is a good start.



10. Competitive analysis vs Codex and Claude Code

The comparison uses publicly documented behaviour of OpenAI Codex (the CLI and its cloud agent) and Anthropic Claude Code as of September 2026, set against COMU as read in source. The aim is not to declare a winner but to locate, for each dimension, the concrete architectural reason COMU is ahead or behind, and to separate observed fact from inference.

CAPABILITY	COMU	CODEX	CLAUDE CODE
Planning	Fixed templates by keyword; not model authored	Model driven plan and TODO tracking in the agent loop	Model driven planning; explicit plan mode; TODO tool
Context management	None on the wire; engine unused; overflow fails	Managed context with compaction over long sessions	Automatic context compaction; rolling summaries
Repo understanding	Text scan + tree; no index	Reads files in loop; AGENTS.md conventions	Reads files; CLAUDE.md memory; agentic search
Sub agents / parallelism	Read only workers, depth 1, synchronous	Cloud can run parallel tasks	First class subagents and agent teams; parallel
Sandbox / isolation	App level allowlist; no OS sandbox; server unauth	OS sandbox (Seatbelt / Landlock, seccomp)	Permission prompts; configurable allow and deny; hooks
Permission model	Contract unenforced; approvals unwired	Approval modes: suggest, auto edit, full auto	Per tool permissions; ask, allow, deny; hooks
Model routing / fallback	Single provider; no fallback	OpenAI models; effort levels	Model selection; Anthropic family
Extensibility	No MCP; fixed tool set	MCP; AGENTS.md; SDK harness	MCP; hooks; skills; subagent configs
Resumability	None; in memory only	Session resume; cloud persistence	Session continue and resume
Model call reliability	Backoff, budget, taxonomy (strong)	Robust	Robust
Transparency UI	Seven tab workspace; event stream (ahead)	Terminal + IDE + cloud UI	Terminal + IDE; rich status
Verify / repair loop	Built in verify, diagnose, repair (distinctive)	Runs tests; iterates	Runs tests; iterates; hooks for gates

10.1 The architectural reasons behind the gaps

- Planning gap.** Codex and Claude Code let the model author and revise the plan and track progress against it turn by turn; COMU's planner is a deterministic template chooser and the plan never enters the prompt. The gap is not model quality; it is that COMU removed the model from planning. This is the single largest capability differentiator and it is squarely fixable.
- Context gap.** Both competitors compact context to sustain long sessions; COMU sends full history and has no compaction, so it cannot sustain long sessions at all. The reason is that COMU's context engine was built as a UI feed rather than a prompt builder.
- Sandbox gap.** Codex confines execution with OS primitives; COMU relies on an application allowlist and, on the runtime perimeter, has no auth at all. COMU's tool guards are good, but the trust boundary is weaker because it is not enforced by the OS or the process perimeter.
- Extensibility gap.** Both competitors support MCP and repository convention files (AGENTS.md, CLAUDE.md); COMU has a fixed internal tool set and no plugin or MCP surface, which caps how far the community and power users can extend it.
- Permission gap.** Both competitors gate actions with real approval modes; COMU built the gate and did not connect it, so it is functionally full auto while presenting as supervised.

10.2 Where COMU is genuinely ahead or even

- Transparency surface.** The seven question framing and the workspace UI are more explicit about what the agent is doing than either competitor's default surface. This is a real, defensible differentiator if the underlying data becomes trustworthy.
- Structured verify, diagnose, repair as a first class engine.** Competitors iterate on tests inside the model loop; COMU has an explicit, budgeted, fingerprinted remediation engine with anti thrash guarantees. Once diagnosis is multi language, this is a distinctive strength.
- Model call reliability layer.** The dedicated request manager is on par with mature agents and better than many open source ones.
- Model agnostic BYOK.** A clean capability profile abstraction that sanitises requests per provider is a solid foundation for genuine multi provider support once routing and fallback are built.

Strategic read. COMU is not trying to beat Codex and Claude Code at raw model horsepower; it cannot, since it runs the same third party models. Its plausible edge is being the most transparent, most inspectable, most controllable coding agent, with a verification engine that produces evidence a team can trust. To earn that position it must first close the correctness, context, planning, and permission gaps that currently make the transparency surface narrate an unreliable process.

11. Claim verification and what is real

The README makes specific claims. Being precise about which are true, which are hollow, and which are unverifiable is essential for an honest roadmap and for user trust.

CLAIM	VERDICT	EVIDENCE
22 modular workspace packages, pnpm monorepo	TRUE	OBSERVED 21 workspace package directories under apps, packages, providers, tools; typed and separated.
32 test files, 311 tests passing	CREDIBLE	OBSERVED 36 test files with about 325 it/test blocks including a 35 case startup performance suite. The count is real; pass status not run here UNVERIFIED .
Provider neutral model gateway; NVIDIA, OpenAI, Ollama, Astra	PARTLY	OBSERVED NVIDIA and OpenAI compatible are real. Ollama is listed but task execution falls back to NVIDIA with a dummy key. Astra is an OpenAI compatible profile pointing at an unverifiable endpoint.
Up to 1,050,000 tokens context (GPT-6 Astra)	UNVERIFIED / HOLLOW	OBSERVED The figure is a profile constant; nothing manages or bounds context, and the loop overflows without compaction. The model and endpoint (api.experiential.com, "GPT-6 Astra") are not a known public model. UNVERIFIED
Context Engine and WorkingSet	HOLLOW	OBSERVED The engine exists but is never called on the execution path; WorkingSet feeds only the UI.
Interactive Human in the Loop cards; approve actions	HOLLOW	OBSERVED InteractionManager and the full UI relay exist, but the orchestrator never raises an approval; writes and commands auto execute.
Automated Verification Matrix	REAL	OBSERVED Verification policy and engine run real validators and aggregate results, subject to the allowlist and language gaps noted.
Completion Gate and Root Diagnosis	REAL	OBSERVED Both exist and function; the gate can misfire on ecosystems without runnable checks.
Instant first paint under 20 ms; non blocking hydration	UNVERIFIED	The webview posts state immediately and there is a perf test suite, but a sub 20 ms paint is a runtime claim not measured here.
Model agnostic, BYOK, keys never exposed to webview	TRUE	OBSERVED Keys live in VS Code SecretStorage; the webview never receives them; requests go direct to the provider.
Signature Activity Timeline and real time state	TRUE	OBSERVED A rich, normalised event stream renders live in the UI.

Trust recommendation. Before wider release, align the README with the implementation: either connect the hollow claims (context engine, approvals, Ollama) or scope them down to "planned." Shipping "approve actions" and "local Ollama" that do not work is the fastest way to lose the trust the transparency thesis is trying to build, and the unverifiable "GPT-6 Astra 1.05M" claim invites scepticism about the credible parts. Under promise here; the real engineering is strong enough to stand without the inflated headline.

12. Prioritised issue matrix

Every issue below carries an id, severity, the affected components, the technical root cause, how it shows up in a real workflow, the recommended fix, implementation difficulty (S, M, L), dependencies, and the validation that proves it fixed. The summary table is followed by detailed cards for the critical blockers.

ID	ISSUE	SEVERITY	EFFORT	COMPONENT
C1	Runtime uses process.cwd() as workspace root; ignores the workspace the extension sends	CRITICAL	S	agent-runtime/server
C2	No context budgeting or compaction; unbounded history overflows and hard fails	CRITICAL	L	orchestrator, context-engine, model-core
C3	Permission contract never enforced at the tool layer; ASK/PLAN can write and execute	CRITICAL	M	orchestrator, tool-core
C4	Approval interactions never raised; autonomous writes despite HITL promise	CRITICAL	M	orchestrator, interaction-manager
C5	Unauthenticated runtime on all interfaces with open CORS	CRITICAL	S	agent-runtime/server, extension
C6	Ollama advertised but runs NVIDIA with a dummy key	CRITICAL	M	agent-runtime/server, providers
C7	Task state in memory only; no durability or resume; 5 minute cap	CRITICAL	L	agent-runtime, event store
H1	Planning is template selection, not model authored; plan not fed to model	HIGH	M	planning-engine, orchestrator
H2	UI Mode selection dropped before reaching the runtime	HIGH	S	chat-provider, protocol, kernel
H3	CHAT mode returns a hardcoded greeting; no conversational model call	HIGH	S	agent-kernel
H4	git_push approval is a self set boolean, not a human gate	HIGH	S	tools/git, interaction-manager
H5	Terminal allowlist too narrow (no make, bash, project scripts) for real repos	HIGH	M	tools/terminal
H6	Validation resolver emits linters the terminal policy denies; Java unmapped	HIGH	S	tools/validation, terminal
H7	Diagnosis evidence parser is JS/TS only; repair loop degraded for other langs	HIGH	M	diagnostics-engine
H8	No model routing or fallback despite the declared interface	HIGH	M	model-core
H9	No MCP or plugin extensibility; fixed tool set	HIGH	L	tool-core, runtime
M1	Completion gate misfires when no runnable checks exist for the ecosystem	MEDIUM	M	verification-engine, orchestrator
M2	Tool calls executed sequentially even when independent and read only	MEDIUM	M	orchestrator
M3	Mutation error escalates to hard task failure instead of model recovery	MEDIUM	S	orchestrator
M4	Terminal boundary uses prefix startsWith; sibling prefix bypass	MEDIUM	S	tools/terminal
M5	shell:true on Windows weakens the no shell posture	MEDIUM	M	tools/terminal
M6	No OS level sandbox for command execution	MEDIUM	L	tools/terminal, runtime
M7	Search has no ripgrep and no semantic index; slow on large repos	MEDIUM	M	tools/search
M8	Memory retrieval is lexical, one shot at task start	MEDIUM	M	memory-engine, orchestrator
M9	No binary or image file handling; no delete or move tool	MEDIUM	S	tools/filesystem
L1	Shipped Boot diagnostic bar in production UI	LOW	S	webview
L2	Double file read per mutation inflates tool traffic and events	LOW	S	orchestrator
L3	Event ids use Date.now() plus random; collisions possible under load	LOW	S	protocol, all emitters
L4	Auto commit message is a crude single file heuristic	LOW	S	orchestrator
L5	web_docs miscategorised as execute rather than network capability	LOW	S	tools/web-docs

12.1 Critical blocker detail cards

C1 · Wrong working directory

CRITICAL

Root cause	OBSERVED In the runtime task handler, <code>const workspaceRoot = resolve(process.cwd())</code> . The extension does send <code>workspace</code> in the task request (from <code>getWorkspaceContext</code>), but the server never reads <code>taskReq.workspace</code> . The server is spawned by the extension with no <code>cwd</code> set, so it inherits the extension host's working directory, which is not reliably the user's project.
Manifestation	The agent reads and writes files relative to the wrong directory. In a normal install it may operate on the VS Code install directory or an unrelated path, so tasks either find nothing, fail the completion gate, or, in the worst case, write files outside the intended project.
Fix	Use <code>taskReq.workspace.rootPath</code> as the workspace root (validated and normalised), and pass it into <code>OrchestratorContext.workspaceRoot</code> . Optionally also set the server <code>cwd</code> , but the request field is authoritative and multi root safe.
Difficulty / deps	S. No dependencies. This is a few lines and should be first.
Validation	Run a task against a fixture repo at a known path; assert every file read and write and command cwd resolves under that path; add a test that a task request with an explicit workspace never touches <code>process.cwd()</code> .

C2 · No context budgeting or compaction

CRITICAL

Root cause	OBSERVED The orchestrator appends every assistant turn and every tool result to <code>messages</code> and sends the full array each iteration. Tool results are not truncated at the history level (a single file read can be up to 1 MB). The <code>ContextEngine.compile()</code> that would curate context is never invoked.
Manifestation	INFERENCE On a task with more than a handful of file reads or a long test log, the request exceeds the model window; the provider returns 400; the request manager treats it as non retryable; the task fails. Effective ceiling is a short task.
Fix	Route every model call through a context builder: token budget per provider, rank and include working set by relevance and recency, summarise or evict the oldest tool outputs, and maintain a compact rolling task state (goal, plan, current step, key findings) that always survives. Reuse the existing <code>compile()</code> and <code>WorkingSetManager</code> as the basis.
Difficulty / deps	L. Depends on H1 (so the rolling state includes the real plan) but can begin independently.
Validation	A task that reads 50 files and runs a long test suite completes without overflow; assert request token count stays under budget across the whole task; measure context efficiency (see KPIs).

C3 + C4 · Authorisation inert (contract unenforced, approvals unraised)

CRITICAL

Root cause	OBSERVED The orchestrator calls <code>executor.execute()</code> directly, bypassing <code>processModelToolCall</code> which is the only path that validates the <code>TaskContract</code> ; and it hands tools a context with write and execute set to ALLOW regardless of mode. <code>InteractionManager.requestApproval</code> exists and is fully wired to the UI but is never called before a mutation or command.
Manifestation	ASK and PLAN modes can still modify files and run commands; the promised approval cards never appear; the agent is effectively full auto while presenting as supervised.
Fix	Route tool calls through the contract validating path; derive tool context permissions from the contract (read only for ASK and PLAN). Introduce an autonomy level (read only, ask, auto); in ask mode, before any write or execute tool, raise an approval interaction and block on the response using the existing <code>InteractionManager</code> and UI.
Difficulty / deps	M. Depends on H2 (mode must actually reach the runtime) for the autonomy selector to be meaningful.
Validation	An ASK task attempting a write is rejected with a contract error; an AGENT task in ask mode pauses at <code>WAITING_FOR_USER</code> before the first write and resumes on approval; a denied approval aborts cleanly.

C5 · Unauthenticated runtime perimeter

CRITICAL

Root cause	OBSERVED <code>app.listen(port)</code> binds all interfaces; <code>cors()</code> is unrestricted; no route checks any token. The task, memory, and cancel routes can be driven by any client that reaches the port.
Manifestation	INFERENCE Any local process (and, on a permissive firewall, any LAN host) can create tasks that read and write workspace files and run allowlisted commands, and can read or write the memory store.
Fix	Bind <code>127.0.0.1</code> ; generate a per session token in the extension, pass via environment to the spawned server, require it on every route (constant time compare); restrict CORS to the webview origin.
Difficulty / deps	S. No dependencies.
Validation	A request without the token is rejected 401; a request from a non loopback address is refused; the extension continues to work with the token.

C6 · Ollama is advertised but not implemented

CRITICAL

Root cause	OBSERVED The provider list marks Ollama as always connected and local, but the task runner's provider selection has no Ollama branch; a local model id falls to the else branch and constructs an NVIDIA provider with a dummy key.
Manifestation	Selecting the local option produces an authentication failure against NVIDIA, not a local run. The offline privacy value proposition is unmet.
Fix	Implement an Ollama provider (OpenAI compatible local endpoint) or remove the option until it exists. Given the OpenAI compatible provider already exists, pointing it at the local Ollama endpoint is a modest amount of work.
Difficulty / deps	M. No hard dependencies; benefits from H8 routing.
Validation	A task with a local model runs against a local Ollama instance end to end and never contacts NVIDIA.

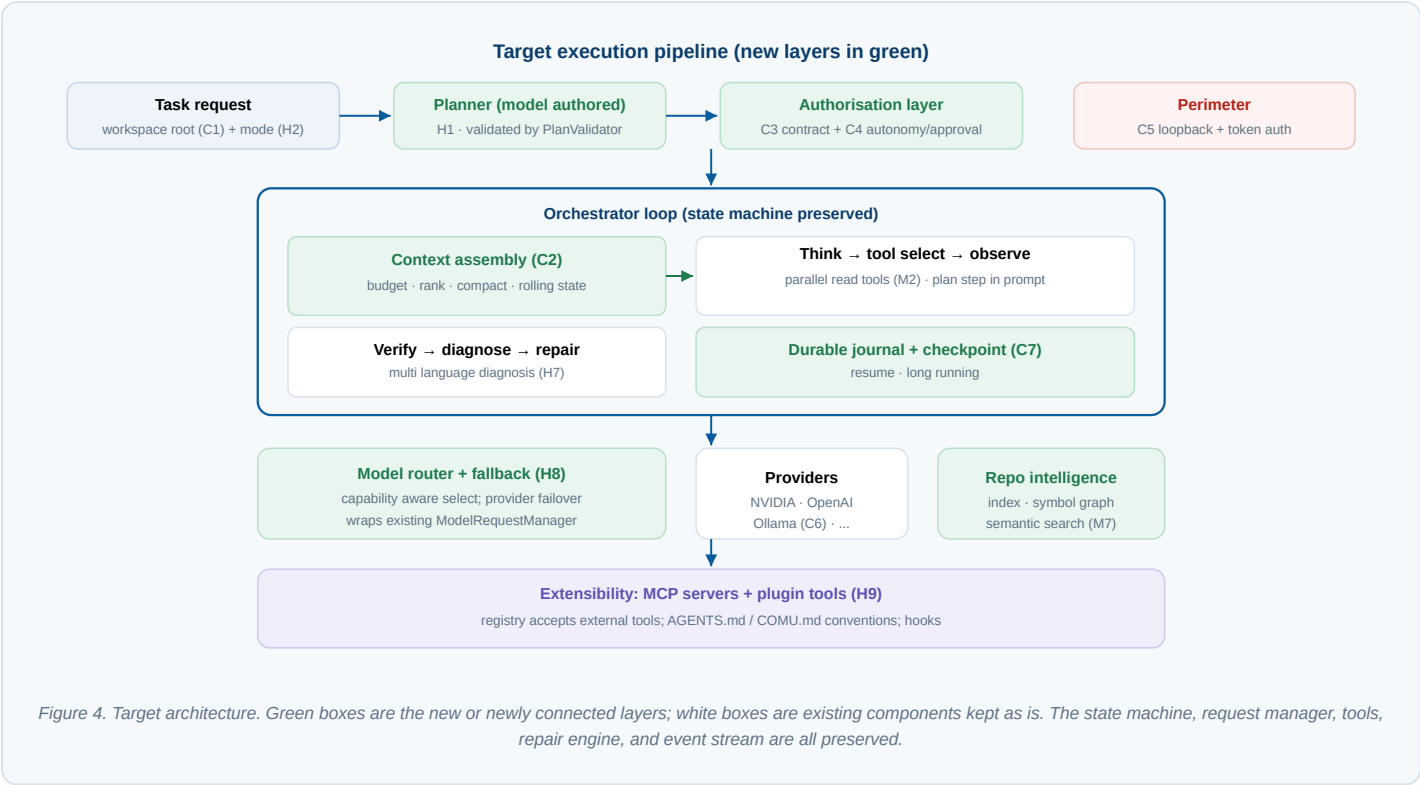
C7 · No durability or resume

CRITICAL

Root cause	OBSERVED The event store is in memory, capped at 5000 events, and cleared five minutes after a task ends. There is no task journal, no checkpoint, and a 5 minute execution cap. A runtime restart loses everything.
Manifestation	Tasks cannot be resumed, re opened, or continued; a crash mid task loses the work and the history; long autonomous tasks are impossible within the cap.
Fix	Persist the event journal and a compact task state to disk (per workspace, alongside memory). Add checkpoint and resume; raise or make the execution cap configurable; expose recent tasks in the UI.
Difficulty / deps	L. Pairs naturally with C2's rolling state object.
Validation	Kill the runtime mid task and resume to the last checkpoint; re open a completed task's history after restart.

13. Proposed target architecture

The target is not a rewrite. It keeps every strong component (request manager, tools, state machine, plan validator, repair engine, memory, event stream, UI) and inserts the missing layers: a context assembly layer between the loop and the model, a real planning call, an enforced authorisation layer, durable task state, and a routing and fallback layer in front of providers. The changes are additive and mostly wiring.



13.1 Advanced capabilities to develop next

Distinguished into foundational (needed to reach parity and reliability) and experimental (differentiation bets, after the foundation is sound).

Foundational

- Repository index: symbol and import graph, definition and reference lookup, embeddings for semantic search.
- AST aware editing for safe multi file refactors (rename a symbol across files with confidence).
- Real subagent orchestration: a planner and executor split, parallel investigation workers, and result aggregation.
- MCP support so external tools and data sources plug in; repository convention file (a COMU.md or AGENTS.md).
- Test driven execution: write or run a failing test first, then implement until it passes, using the existing verify and repair engines.

Experimental (differentiation)

- Evidence bundles: every completed task ships a signed record of what was verified, with logs, for team trust and audit.
- Confidence estimation and self verification: the agent scores its own certainty and escalates to the human when low.
- Issue to pull request workflow with automated code review by a dedicated review subagent.
- Benchmark driven self improvement: the agent runs its own regression suite (section 14) and reports capability deltas per release.
- Dependency and security analysis agents; performance profiling agent; architecture analysis agent as specialised workers.

14. Benchmark and evaluation framework

COMU needs a repeatable, quantitative benchmark it can run every release to prove that engine changes actually improve behaviour rather than just changing it. The framework below defines tiered tasks against fixture repositories, precise success criteria, and a fixed metric set. It is designed to run headless against a pinned model so that changes in COMU, not the model, are what the numbers measure.

14.1 Task tiers

TIER	TASK CLASS	EXAMPLE	PRIMARY SUCCESS CRITERION
T0	Trivial edit	Rename a constant; fix a typo in a string	Exact expected file diff; no other files changed
T1	Single file bug fix	Fix an off by one that a provided test catches	Target test passes; no regressions
T2	Feature addition	Add an endpoint or a function with new tests	New behaviour works; new and existing tests pass
T3	Multi file refactor	Rename a symbol used across five files	All references updated; typecheck and tests pass
T4	Failing test debug	A repo with three failing tests and a subtle cause	All targeted tests pass; root cause is the intended one
T5	Dependency upgrade	Bump a library across a breaking change	Build and tests pass after adapting call sites
T6	Security fix	Patch an injection or path traversal with a test	Exploit test now fails to exploit; suite green
T7	Unfamiliar repo onboarding	Answer architecture questions on a new repo	Answers match a rubric; correct files cited
T8	Issue to implementation	Implement from a written issue with acceptance tests	Acceptance tests pass; scope respected
T9	Long horizon autonomous	A multi step feature needing 20+ tool calls	Completes within budget; verified; minimal human input

Each tier should include at least three fixtures across at least two ecosystems (a TypeScript repo and a Python repo minimum, to exercise the multi language gaps), pinned by commit, with a golden solution and an automated grader (test runner plus a diff or rubric check). Run each fixture N times (say 5) to capture variance.

14.2 Metrics (recorded per run, aggregated per tier and per release)

METRIC	DEFINITION	TARGET TREND
Task completion rate	Fraction reaching COMPLETED with the success criterion met	Up
Functional correctness	Grader pass (tests plus rubric), independent of self reported status	Up
Test pass rate	Fraction of targeted tests passing after the task	Up
Regression rate	Fraction of runs that break a previously passing test	Down
Unnecessary change count	Files or hunks changed beyond the golden solution scope	Down
Human intervention count	Approvals or clarifications required per task	Down (at fixed safety)
Tool call efficiency	Tool calls per completed task; redundant reads flagged	Down
Token consumption	Prompt plus completion tokens per task (provider usage)	Down
Context efficiency	Useful tokens (files touched) divided by tokens sent	Up
Execution time	Wall clock per task	Down
Recovery success rate	Fraction of failed verifications the repair loop fixes	Up
Planning accuracy	Do plan steps match the work actually needed (graded)	Up
Patch quality	Reviewer or rubric score of the final diff	Up

14.3 Failure taxonomy (every run labelled)

Wrong directory, context overflow, planning miss, tool error unrecovered, verification unavailable, gate misfire, infinite or truncated loop, unnecessary changes, regression introduced, cancelled, provider error. Tracking the distribution over releases shows whether a fix moved the needle on its target failure class without creating another.

How to use it. Freeze the model and fixtures; run the suite before and after each engine change; require that a change improves its target metric without regressing others beyond a tolerance. Publish the per release table (see KPIs). This is what turns "we refactored planning" into "planning accuracy rose from 0.3 to 0.7 and token consumption fell 22 percent on T2 through T4."

15. Phased advancement roadmap

Nine phases across roughly 6 to 18 months. Phases 0 and 1 are sequential and blocking; later phases parallelise (section 16). Each phase lists its problem, target, approach, affected modules, difficulty, risks, validation, and success criterion.

Phase 0 · Stabilisation (weeks 1 to 3) — nothing else ships first

ITEM	TARGET AND APPROACH	VALIDATION / SUCCESS
C1 workspace root	Use the request's workspace root; drop <code>process.cwd()</code> .	Fixture task confined to the given path; regression test.
C5 perimeter auth	Loopback bind; per session token via env; scoped CORS.	Unauthenticated request rejected; extension still works.
C6 Ollama honesty	Implement local provider or remove the option and the claim.	Local task runs locally, or option absent.
H2 mode plumbing	Send UI mode; honour it; skip re classification when explicit.	Selecting AGENT forces AGENT; test asserts pass through.
H3 real chat	Make CHAT call the model with a light system prompt.	A greeting yields a real response, not a fixed string.
L1 debug bar	Remove or dev gate the Boot diagnostic strip.	Not present in a packaged build.

Risk: low; these are contained fixes. **Difficulty:** mostly S. **Phase success:** a real task on a real repo runs in the correct directory, honours the chosen mode, and the runtime is not open to other processes.

Phase 1 · Authorisation and safety (weeks 3 to 6)

Problem: C3 and C4; the agent is full auto behind a supervised facade. **Target:** enforce the contract at the tool layer; add autonomy levels; raise real approval cards before write and execute in ask mode. **Approach:** route calls through the contract validating path; derive permissions from the contract; wire the existing InteractionManager into the loop before mutations. **Modules:** orchestrator, tool-core, interaction-manager, webview. **Risk:** approval UX friction; mitigate with "approve for session." **Validation:** ASK write rejected; ask mode pauses before write; deny aborts cleanly. **Success:** COMU is as safe as it looks.

Phase 2 · Context and reasoning (weeks 5 to 12)

Problem: C2; no context management, planning is a template. **Target:** real model authored plans (H1) fed to the model per step; a context assembly layer with token budget, ranking, compaction, and a rolling task state. **Approach:** add a planning model call validated by `PlanValidator`; make `compile()` the prompt source; introduce per provider token budgets and summarisation. **Modules:** planning-engine, context-engine, orchestrator, model-core. **Risk:** summarisation losing detail; mitigate with a preserved rolling state and pinned working set. **Validation:** the 50 file task completes; planning accuracy and context efficiency measured on the benchmark. **Success:** long tasks stop overflowing; plans reflect the actual work.

Phase 3 · Tool system and verification hardening (weeks 8 to 14)

Problem: H5, H6, H7, M1, M4, M5, M9; narrow allowlist, denied linters, JS only diagnosis, gate misfires, boundary and shell edges, no binary or delete tools. **Target:** project script execution under policy, coherent multi language verify and diagnose, correct boundary checks, safe file operations. **Approach:** allow project defined scripts through a reviewed path; align the validation resolver with the policy allowlist; extend the evidence parser to Python, Go, Rust; unify the boundary helper; add delete and move and binary aware reads. **Modules:** terminal, validation, diagnostics-engine, filesystem, verification-engine. **Risk:** loosening the allowlist; mitigate by gating new executables behind approval. **Validation:** a Python repo task verifies, diagnoses, and repairs end to end. **Success:** COMU works beyond TypeScript.

Phase 4 · Durability and resumability (weeks 12 to 18)

Problem: C7; ephemeral state, 5 minute cap. **Target:** durable task journal and checkpoints; resume; configurable long running budgets; task history in the UI. **Approach:** persist events and rolling state to disk per workspace; checkpoint at step boundaries; resume from the last checkpoint. **Modules:** agent-runtime, event store, webview. **Risk:** journal size; mitigate with compaction and retention. **Validation:** kill and resume mid task; re open completed tasks after restart. **Success:** genuinely long tasks and no lost work.

Phase 5 · Model routing and reliability (weeks 14 to 20)

Problem: H8; single provider, no fallback. **Target:** capability aware routing and cross provider fallback wrapping the existing request manager. **Approach:** implement `ModelRouter`; on non recoverable provider failure, fail over to a configured alternate; route by task class and cost. **Modules:** model-core, runtime. **Risk:** profile mismatches across providers; mitigate with the existing capability sanitiser. **Validation:** injected provider outage triggers fallback and the task still completes. **Success:** resilience to provider issues; cost aware selection.

Phase 6 · Repository intelligence (weeks 18 to 30)

Problem: M7, M8; no index, lexical memory. **Target:** symbol and import graph, semantic search, AST aware editing, semantic and live memory retrieval. **Approach:** build an incremental index; add embeddings backed search; use the graph for reference safe refactors; make memory a per turn context participant. **Modules:** new indexing package, search, context-engine, memory-engine. **Risk:** index maintenance cost; mitigate with incremental updates. **Validation:** T3 refactor and T7 onboarding scores rise. **Success:** COMU understands repositories rather than reading them blindly.

Phase 7 · Extensibility and agent teams (weeks 24 to 40)

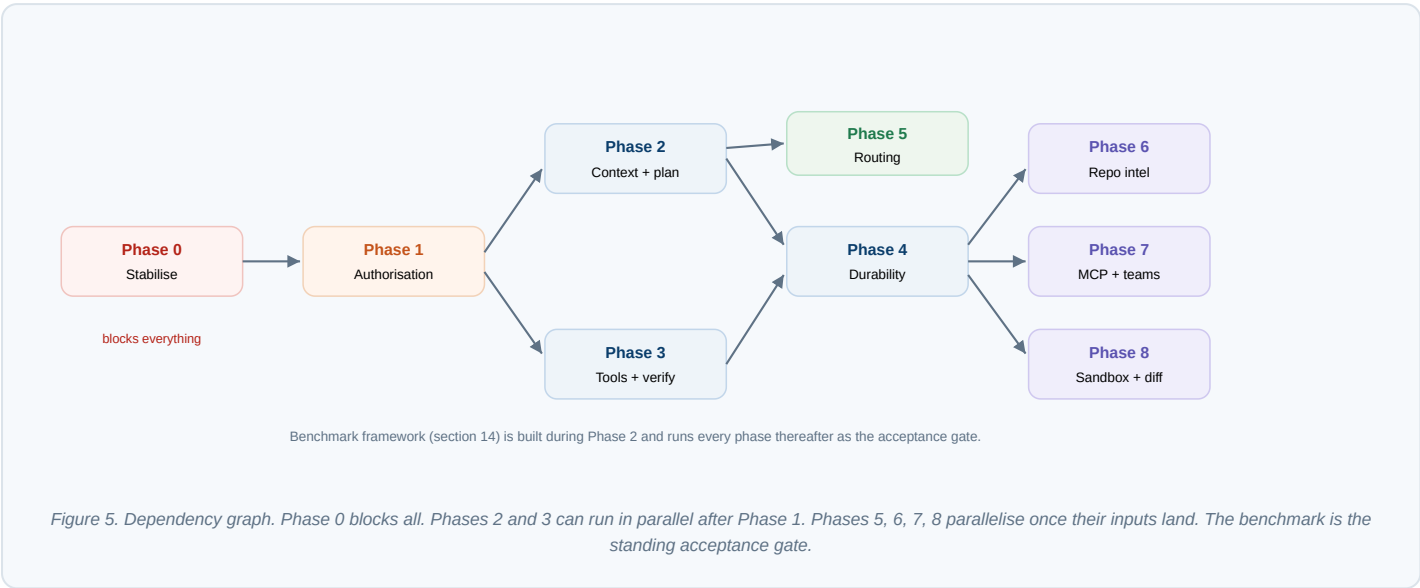
Problem: H9 and shallow subagents. **Target:** MCP support, a convention file, hooks, and real subagent orchestration (planner and executor, parallel workers, specialised review, test, and security agents). **Approach:** open the tool registry to external MCP tools; add lifecycle hooks; extend the subagent manager to parallel implementation and aggregation. **Modules:** tool-core, runtime, agent-core. **Risk:** concurrency and safety of parallel writers; mitigate with per worker scopes and the authorisation layer. **Validation:** an MCP tool is invoked; a review subagent comments on a diff. **Success:** a platform, not a fixed tool.

Phase 8 · Differentiation and OS sandbox (weeks 30 to 52+)

Problem: M6 and the differentiation thesis. **Target:** OS level sandbox for command execution; evidence bundles; confidence estimation; issue to pull request; benchmark driven release reporting. **Approach:** platform sandboxes (Landlock or seccomp, Seatbelt, job objects); signed task evidence; a self evaluation harness. **Modules:** terminal, runtime, new evidence and eval packages. **Risk:** platform specific sandbox complexity. **Validation:** a hostile command is confined; every release ships a benchmark delta. **Success:** the most transparent, controllable, and self measuring coding agent.

16. Dependency map and parallelisation

What must precede what, and what can run in parallel. The critical path runs through correctness, then authorisation, then context and reasoning; tool hardening, durability, and routing can proceed alongside once Phase 0 lands.



CAN RUN IN PARALLEL	WHY
Phase 2 (context and plan) and Phase 3 (tools and verify)	Different modules; both depend only on Phase 0 and 1.
Phase 4 (durability) and Phase 5 (routing)	Independent subsystems; both benefit from Phase 2's rolling state but do not block on it.
Phases 6, 7, 8	Additive platform work once the core loop is correct, safe, and durable.

Hard ordering: C1 before anything (correct directory is the precondition for meaningful testing); Phase 1 before the autonomy selector is meaningful (which needs H2 from Phase 0); H1 planning before or with C2 context (the rolling state should carry the real plan); H7 multi language diagnosis before claiming non JS support.

17. Measurable KPIs

Targets are directional milestones against the benchmark (section 14) at a pinned model. Baselines are the current state; several are effectively zero or not measurable today because the relevant path fails.

KPI	BASELINE	PHASE 2 EXIT	PHASE 4 EXIT	PHASE 8 EXIT
T1 to T2 completion rate	low / unstable	0.60	0.75	0.88
T3 to T4 completion rate	~0	0.35	0.55	0.78
T9 long horizon completion	0 (5 min cap)	0.20	0.45	0.70
Functional correctness (graded)	not measurable	0.62	0.76	0.88
Regression rate	unknown	<0.15	<0.10	<0.05
Context overflow failures	frequent	<0.05	<0.02	<0.01
Planning accuracy (graded)	~0.2 (template)	0.65	0.75	0.85
Tool calls per completed task	n/a	baseline	-15%	-30%
Tokens per completed task	n/a	baseline	-20%	-35%
Context efficiency	very low	0.35	0.50	0.65
Recovery success rate (repair)	JS/TS only	0.45	0.60	0.75
Unsafe auto actions	all writes auto	0 in ask mode	0	0
Mean time to resume after crash	infinite (no resume)	n/a	<10 s	<5 s
Non JS ecosystem parity (T* on Python)	degraded	partial	0.60 of JS	0.90 of JS

Process KPI. Every release publishes the benchmark table with deltas. A change that does not move its target KPI without regressing others beyond tolerance does not merge. This converts the roadmap from a list of intentions into a measured programme.

18. Differentiation thesis and closing

COMU shares its models with Codex and Claude Code, so it will not win on raw model capability. Its credible, defensible position is to be **the coding agent a team can trust and inspect**: the most transparent about what it did, the most controllable about what it is allowed to do, and the only one that ships verifiable evidence of what it checked. The seven question workspace, the structured verify and repair engine, and the event sourced timeline are the seeds of that position. They are, today, a beautiful dashboard over an unreliable process. The entire near term programme is about making the process worthy of the dashboard.

Transparency Keep and deepen the event stream and the workspace. Add cost and token visibility, task history, and exportable activity logs. Nobody else surfaces the agent's internal state this well.	Control Deliver the approval model that is already built but unwired. Autonomy levels, per action approval, and read only modes that are actually enforced. This is a trust feature enterprises pay for.	Evidence Turn the verification engine into signed evidence bundles: a record of exactly what was tested, diagnosed, and repaired, per task. This is a genuinely novel artifact for team and audit contexts.
--	--	---

The three sentence recommendation

1. **Stop and stabilise.** Close the seven critical blockers (starting with the working directory and the perimeter) before writing any new feature; today the product likely operates on the wrong files and is open to any local process.
2. **Connect what you built.** The context engine, the approval flow, the plan validator, the repair engine, and the model reliability layer are good; the highest leverage work is wiring them into the loop and replacing the template planner with a real one, not building anything new.
3. **Measure everything.** Stand up the benchmark in Phase 2 and gate every change on it, so the roadmap produces proven capability rather than more surface area.

Final note, kept honest. The quality gap here is not a talent gap. The components that are finished (the request manager, the filesystem and terminal safety, the plan validator, the diagnostics fingerprinting, the repair budgeting, the memory store, the event UI) are the work of a team that can clearly build hard things well. The issue is purely one of sequencing: too much surface was shipped as presentation ahead of the engine that would make it real. Reverse that order (engine first, claims second) and COMU has a credible path to a differentiated, trustworthy coding agent rather than an impressive demo.